Glossary

aggregate

A collection of objects that are managed as a single entity, typically referring to real-world concepts. A concept from DDD.

Amazon Web Services (AWS)

The public cloud offering from Amazon.

API gateway

A component that normally sits on the perimeter of a system and routes calls from external sources (such as user interfaces) to microservices, amongst many other things.

authentication

The process whereby a principal proves that they are who they say they are. This could be as simple as a person providing their username and password.

authorization

The process that determines if an authorized principal is allowed to access a given piece of functionality.

Azure

The public cloud offering from Microsoft.

backend for frontend (BFF)

A server-side component that provides aggregation and filtering for a specific user interface. An alternative to a general-purpose API gateway.

bounded context

An explicit boundary within a business domain that provides functionality to the wider system, but which also hides complexity. Often maps to organizational

boundaries. A concept from DDD.

bulkhead

A part of the system in which a failure can be isolated, so that the rest of the system can keep operating even if a failure occurs.

choreography

A style of saga, where responsibility for what should happen when is distributed across multiple microservices, rather than managed by a single entity.

circuit breaker

A mechanism placed around a connection to a downstream service that can allow you to fail fast if the downstream service is suffering from problems.

cohesion

The extent to which the code that changes together stays together.

collective ownership

A style of ownership in which any developer is allowed to change any part of the system.

container

A package of code and dependencies that can be run in an isolated way on a machine. Conceptually similar to virtual machines, although much more lightweight.

continuous delivery (CD)

A delivery approach in which you explicitly model the path to production, treat every check-in as a release candidate, and can easily assess the suitability of any release candidate to be deployed into production.

continuous deployment

An approach wherein any build that passes all automated steps is automatically deployed into production.

continuous integration (CI)

The regular (daily) integration of changes with the rest of the codebase, along with a suite of tests to validate that the integration has worked.

Conway's law

: The observation that the communication structures of organizations end up driving the design of the computer systems that these organizations build.

coupling

: The extent to which changing one part of the system requires a change in another. Low coupling is normally desirable.

cross-functional requirement (CFR)

: A general property of the system, such as the required latency for operations, security of data at rest, etc. Also known as nonfunctional requirement (but I greatly prefer cross-functional as a description).

customizable off the shelf software (COTS)

: Third-party software that is heavily customized by the end user and is also normally run on their own infrastructure. Typical examples include content management systems and customer relationship management platforms.

data partitioning

: Scaling a system by distributing load based on some facet of the data. For example splitting load based on customer or product type.

detective controls

: A security control that will help you identify if an attack is underway/has happened.

domain coupling

: A form of coupling where one microservice is "coupled" to the domain protocol exposed by another microservice.

domain-driven design (DDD)

: A concept whereby the fundamental problem/business domain is explicitly modeled in the software.

Docker

: A set of tooling to help build and manage containers.

enabling team

A team that supports stream-aligned teams in doing their job. Typically, an enabling team has a specific focus—e.g., usability, architecture, security.

error budget

Relates to the acceptable level to which an SLO can be out of bound, typically defined in an acceptable degree of downtime for a service.

event

Something that happens in the system that other parts of the system might care about—for example "Order Placed" or "User Logs In."

feature branching

Creating a new branch for each feature that is being worked on, merging that branch back into mainline once the feature is complete. Something I discourage.

Function as a Service (FaaS)

A type of serverless platform that invokes arbitrary code based on certain types of triggers—for example, launching code in reaction to an HTTP call, or a message being received.

governance

Agreeing how things should be done, and making sure they are done that way.

graceful extensibility

How well we deal with a situation that is unexpected.

GraphQL

A protocol that allows the client to issue custom queries that can result in calls being made to multiple downstream microservices. Useful to aid call aggregation and filtering for external clients without requiring the use of BFF or API gateways.

horizontal duplication

Scaling a system by having multiple copies of a thing.

idempotency

The property of a function where even if it is called multiple times, the result is the same. Useful to allow operations on microservices to be safely retried.

independent deployability

: The ability to make a change to a microservice and deploy it into production without having to change or deploy anything else.

information hiding

: An approach wherein all information is hidden by default inside a boundary, and only the bare minimum is exposed to satisfy external consumers.

infrastructure as code

: Modeling your infrastructure in code form, allowing for infrastructure management to be automated and the code to be version controlled.

JSON Web Token

: A standard for creating a JSON data structure that can be optionally encrypted. It is typically used for transmitting information about authenticated principals.

Kubernetes

: An open source platform that manages container workloads across multiple underlying machines.

library

: A set of code that is packaged in such a way that it can be reused in multiple programs.

lockstep deployment

: The need to deploy two or more things at the same time, because a change has occurred that requires it. The opposite of independent deployability. In general, avoid.

message

: Something sent to one or more downstream microservices via an asynchronous communication mechanism like a broker. Could contain a variety of payloads, such as a request, response, or an event.

message broker

: Dedicated software that manages the asynchronous communication between processes, typically providing capabilities like guaranteed delivery (for some definition of the word *guaranteed*).

microservice

An independently deployable service that communicates with other microservices via one or more communication protocols.

monorepo

A single repository that contains all the source code for all your microservices.

multirepo

An approach in which each microservice has its own source code repository.

orchestration

A style of saga wherein a central unit (aka the orchestrator) manages the operation of other microservices to carry out a business process.

personally identifiable information (PII)

Data that when used in isolation, or in addition with other information, could be used to identify an individual.

preventative control

A security control that aims to stop an attack from happening.

principal

Something—typically a person, though it could also be a program—that is requesting to be authenticated and authorized to gain access.

request

Sent by one microservice to another asking the downstream microservice to do something.

response

Transmitted back as the result of a request.

responsive control

A security control that helps you respond during/after an attack.

robustness

The ability of a system to keep operating even when something bad happens.

saga

A way of modeling long-lived operations in such a way that resources don't need to be locked for extended periods of time. Preferred over distributed transactions when implementing business processes.

serverless

An umbrella term for cloud products that from the user's point of view abstract away the underlying computers, to the extent that the user no longer needs to care about them. Examples of these products include AWS Lambda, AWS S3, and Azure Cosmos.

service-level agreement (SLA)

An agreement between an end user and a service provider (e.g., customer and vendor) that defines the bare minimum acceptable service offering, and the penalties that apply if the agreement isn't met.

service-level indicator (SLI)

A measure of how your system is behaving—for example, a response time.

service-level objective (SLO)

An agreement as to what the acceptable range of a given SLI is.

service mesh

A distributed type of middleware that provides cross-cutting functionality primarily for synchronous point-to-point calls—for example, mutual TLS, service discovery, or circuit breakers.

service-oriented architecture (SOA)

A type of architecture wherein the system is broken apart into services that can run on different machines. Microservices are a *type* of SOA that prioritizes independent deployability.

single-page application (SPA)

A type of graphical user interface wherein the UI is delivered in a single browser pane, without the need for navigation to other web pages.

stream-aligned team

A team focused on the end-to-end delivery of a valuable stream of work. This is a long-lived team that will normally be directly customer focused and cut across data and backend and frontend code.

strong ownership

A style of ownership in which parts of the system are owned by specific teams, and changes to a particular part of the system can be made only by the team that owns it.

sustained adaptability

The ability to continually adapt to changing environments, stakeholders, and demands.

threat modeling

The process in which you understand the threats that might be brought to bear on your system and prioritize which threats need to be addressed.

trunk-based development

A style of development wherein all changes are made directly into the main trunk of the source control system, including changes that aren't yet complete.

ubiquitous language

Defining and adopting a common language to be used in code and in describing the domain, to aid communication. A concept from DDD.

vertical scaling

Improving the scale of the system by getting a more powerful machine.

virtual machine (VM)

An emulation of a machine wherein the machine appears to all intents and purposes to be a dedicated physical machine.

widget

A component of a graphical user interface.

Previous chapter

< Bibliography

Next chapter

Index >